Abschlussbericht

Datenbank-Managementsystem Café Teheran to Tokyo

Projektteam:

Siamak Goudarzi

Kapitel 1: Projektvorstellung

1.1 Geschäftsvorstellung

Das Café *Teheran to Tokyo* ist ein innovatives Premium-Café in Berlin, das eine Brücke zwischen der persischen und japanischen Kaffeekultur schlägt. Das Unternehmen zeichnet sich durch eigene Röstungen, eine internationale Community und ein mehrsprachiges Team aus.

Das Herzstück des Betriebs ist ein CRM-gesteuertes System, das Kundenbindung, Mitarbeiter-Performance, Event-Management und Lagerlogistik auf ein professionelles Niveau hebt.

1.2 Zielsetzung des Datenbanksystems

Das Ziel dieses Projekts war die Entwicklung eines relationalen Datenbanksystems (**CafeCRM_DB**), um:

- den täglichen Verkaufsbetrieb effizient abzuwickeln
- ein dynamisches Loyalty-Programm (Treuepunkte-System) zu verwalten
- die Leistung der Baristas mittels KPI-Analysen zu bewerten
- Lagerbestände und Lieferantenbeziehungen zu optimieren
- kulturelle Events und deren Teilnehmer zu organisieren

Kapitel 2: Systemanalyse und UML-Modellierung

In dieser Phase wurde die Architektur des Systems *Café Teheran to Tokyo* konzeptionell analysiert, um eine konsistente, redundanzfreie und logisch strukturierte Datenbank zu gewährleisten.

2.1 Use Case Analyse (Anwendungsfälle)

Das System bedient drei Hauptakteure:

• Kunde (Customer)

- Tätigt Bestellungen
- Nimmt an kulturellen Events teil
- Sammeln von Treuepunkten

• **Mitarbeiter (Barista)**

- Erfasst Bestellungen
- Dokumentiert Trainingseinheiten (SCA-Zertifizierungen)

• **Manager**

- Führt KPI-Analysen durch
- Überwacht Lagerbestände
- Plant kulturelle Events

2.2 Klassendiagramm und Datenstruktur (High-Level)

Die Datenbank besteht aus **14 Tabellen**, die in funktionale Module gegliedert sind:

Verkaufsmodul

- Order
- OrderItem
- Product
- ProductCategory

CRM-Modul

- Customer
- LoyaltyLevel

Personalmodul

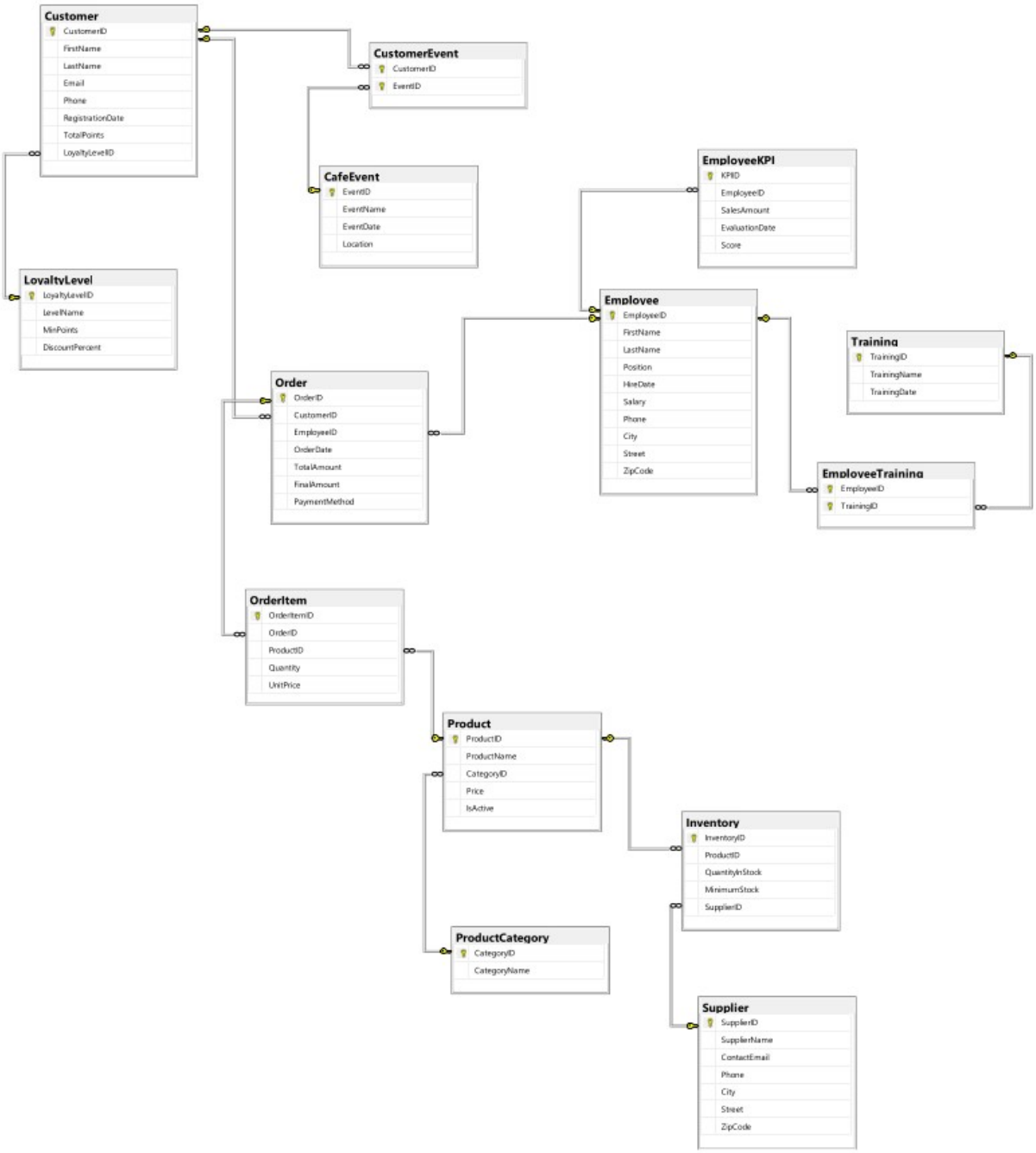
- Employee
- EmployeeKPI
- Training
- EmployeeTraining (n:m)

Lager & Logistik

- Inventory
- Supplier

Event-Management

- CafeEvent
- CustomerEvent (n:m)


Customer
CustomerID
FirstName
LastName
Email
Phone
RegistrationDate
TotalPoints
LoyaltyLevelID
CustomerEvent
CustomerID
EventID
CafeEvent
EventID
EventName
EventDate
Location
EmployeeKPI
KPID
EmployeeID
SalesAmount
EvaluationDate
Score
LoyaltyLevel
LoyaltyLevelID
LevelName
MinPoints
DiscountPercent
Employee
EmployeeID
FirstName
LastName
Position
HireDate
Salary
Phone
City
Street
ZipCode
Training
TrainingID
TrainingName
TrainingDate
Order
OrderID
CustomerID
EmployeeID
OrderDate
TotalAmount
FinalAmount
PaymentMethod
EmployeeTraining
EmployeeID
TrainingID
OrderItem
OrderItemID
OrderID
ProductID
Quantity
UnitPrice
Product
ProductID
ProductName
CategoryID
Price
IsActive
Inventory
InventoryID
ProductID
QuantityInStock
MinimumStock
SupplierID
ProductCategory
CategoryID
CategoryName
Supplier
SupplierID
SupplierName
ContactEmail
Phone
City
Street
ZipCode

2.3 Beziehungen und Kardinalitäten

1:n Beziehungen

- Kunde → Bestellungen
- Bestellung → OrderItems
- Produkt → OrderItems
- Lieferant → Inventory

n:m Beziehungen

- Customer ↔ CafeEvent (über CustomerEvent)
- Employee ↔ Training (über EmployeeTraining)

2.4 Sequenzanalyse: Bestellvorgang (Order Process)

Der Ablauf einer Bestellung folgt einer klar definierten Business-Logik:

1. **Validierung:** Prüfung des Lagerbestands über *Inventory*.
2. **Berechnung:** Rabattermittlung über **fn_GetDiscountRate**.
3. **Transaktion:** Erstellung von Order + OrderItem durch **usp_CreateNewOrder**.
4. **Automatisierung:** Trigger **trg_UpdateStockAfterSale** reduziert den Bestand.
5. **Loyalty-Update:** Punkteberechnung über **fn_CalcLoyaltyPoints**.
6. **Automatisches Loyalty-Upgrade:** Der Trigger trg_CheckLoyaltyUpgrade prüft nach jeder Punkteänderung, ob der Kunde ein höheres Loyalty-Level erreicht hat und aktualisiert es automatisch.

Kapitel 3: Technische Implementierung und Business-Logik

3.1 Validierung durch Constraints

- CHECK (Price > 0) verhindert fehlerhafte Preisangaben
- CHECK (QuantityInStock >= 0) verhindert negative Lagerbestände
- KPI-Scores werden im Bereich 0–100 gehalten

3.2 Automatisierung durch Trigger

- **trg_UpdateStockAfterSale:** Automatische Lagerabbuchung nach jeder Bestellung
- **trg_CheckLoyaltyUpgrade:** Automatische Hochstufung des Loyalty-Levels *(falls implementiert – sonst entfernen)*
- ***trg_UpdateEmployeeKPI (NEU)*** *Erhöht den KPI-Score des Mitarbeiters automatisch nach jeder neuen Bestellung.*

- ***trg_CheckLoyaltyUpgrade:*** *Aktualisiert das Loyalty-Level eines Kunden automatisch, sobald sich seine Gesamtpunkte (TotalPoints) ändern.*

3.3 Gespeicherte Prozeduren und Funktionen

Skalarfunktionen

- **fn_CalcLoyaltyPoints** – Umsatz → Punkte
- **fn_GetDiscountRate** – Rabatt basierend auf LoyaltyLevel

Tabellenwertfunktion

- **fn_InventoryAlert** – Produkte unter Mindestbestand

Stored Procedure

- **usp_CreateNewOrder** – vollständiger Bestellprozess inkl. Rabatt, Punkte, Lagerabbuchung

3.4 Sichten (Views)

vw_SalesSummary
Bestellübersicht mit Kunden- und Mitarbeiterdaten.

vw_EmployeePerformance
Umsatzranking der Mitarbeiter (GROUP BY + HAVING).

vw_InactiveClients
Identifiziert Kunden ohne Bestellungen.

vw_OrderDetails (NEU)
Detaillierte Bestellübersicht:

- OrderID
- OrderDate
- FinalAmount
- CustomerName
- EmployeeName
- PaymentMethod

Kapitel 4: Sicherheit, Testung und Abgabe

4.1 Sicherheitskonzept

Der Benutzer **CafeAppUser** besitzt:

- SELECT
- INSERT
- UPDATE
- EXECUTE

Er darf **keine** strukturellen Änderungen durchführen (DROP, ALTER).

4.2 Testprotokoll

Zur Sicherstellung der Funktionsfähigkeit wurden **9 Tests** durchgeführt:

1. TEST 1- Überprüfung der Basisdaten
2. TEST 2 - Transaktionslogik & Automatisierung.sql
3. TEST 3 - n-m Verknüpfungen (Events & Trainings)
4. TEST 4 - Management-Reporting & Sichten
5. TEST 5 - Validierung der Constraints
6. TEST 6 - Berechtigungsprüfung (Security Test)
7. TEST 7 - Referenzielle Integrität (ON DELETE-Verhalten)
8. TEST 8 - Skalarfunktion 'fn_GetDiscountRate'
9. TEST 9 - (Transaktion) Überprüfung der referenziellen Integrität (Löschverhalten)

4.3 Übergabeformate

Die Abgabe umfasst:

1. **001–DB-Create-Scripts** (Tabellen, Constraints, Indexe)
2. **002–Abfragen und Views**
3. **003–Stored Functions**
4. **004–Stored Procedures**
5. **005–DML-Trigger**
6. **006–Login-User-Role**
7. **007-Tests**